

Why Plotly annotations misalign at intermediate angles on log scales

The core problem is how Plotly handles rotated annotations without arrows: it rotates the text first, then calculates anchors on the expanded bounding box. This creates angle-dependent positioning errors where extreme angles ($\sim 0^\circ$, $\sim 90^\circ$) align correctly while intermediate angles ($\sim 17^\circ$, $\sim 34^\circ$, $\sim 51^\circ$) show increasing offset. The effect compounds on semi-log charts due to asymmetric coordinate scaling.

The bounding box rotation problem explains your exact pattern

When you create an annotation with `textangle` but without an arrow (`showarrow: false`), Plotly executes a **two-step positioning process** that creates predictable misalignment at certain angles:

1. Rotate the text around its center by the specified `textangle`
2. Calculate a new axis-aligned bounding box around the rotated text
3. Apply the anchor (`xanchor`/`yanchor`) to this expanded bounding box [GitHub](#)

At **extreme angles** (0° , $\pm 68^\circ$, $\pm 90^\circ$), one dimension dominates. When text rotates near 0° or 90° , the resulting bounding box remains similar to the original text box dimensions—anchoring remains predictable. But at **intermediate angles** ($\pm 17^\circ$, $\pm 34^\circ$, $\pm 51^\circ$), both sine and cosine have significant non-zero values. A rotated bounding box at 45° expands to roughly **1.41× the original dimensions** in both width and height. The anchor point calculation then places the text at a position on this larger box that doesn't correspond to where you specified.

The mathematical relationship is: $\text{New box width} = |W \cdot \cos(\theta)| + |H \cdot \sin(\theta)|$. At $\theta = 68^\circ$, $\cos(68^\circ) \approx 0.37$ while $\sin(68^\circ) \approx 0.93$ —the height term dominates, making calculations more predictable. At $\theta = 34^\circ$, $\cos(34^\circ) \approx 0.83$ and $\sin(34^\circ) \approx 0.56$ —both contribute significantly, maximizing the bounding box expansion effect.

Shapes bypass this problem entirely

Shapes (lines) in Plotly use a completely different rendering path. They're defined by explicit coordinate pairs (`x0`, `y0`, `x1`, `y1`) and rendered directly as SVG path elements [Plotly](#) without any bounding box calculations, anchor adjustments, or rotation-related positioning logic. This is why your lines appear exactly where specified while annotations using the "same" formula drift.

The source code comment in `set_convert.js` explicitly notes: "For log axes, ranges, tick0, and annotation positions use linearized format. TODO: in v2.0 we plan to change it to data format. At that point shapes will work the same way as ranges, tick0, and annotations." This confirms shapes and annotations currently use different coordinate handling.

The xanchor "auto" behavior adds another layer of variance

With `xref: 'paper'` and no arrow, the default `xanchor: 'auto'` doesn't resolve to `'center'`—it picks **the closest side**. From the official documentation: "If 'auto', the anchor is equivalent to 'center' for data-referenced annotations or if there is an arrow, whereas **for paper-referenced with no arrow, the anchor picked corresponds to the closest side.**" [Plotly](#) [Anvil](#)

This means annotations at different positions may receive different anchor values (left/center/right), causing inconsistent positioning across your radial layout. Combined with the rotated bounding box issue, this creates the complex misalignment pattern you observe.

Paper coordinates also differ between shapes and annotations

A community-reported finding reveals another subtle issue: "The shapes canvas is **inclusive of the area encompassing axis labels and ticks** while annotations does not, which means I cannot easily align a shape and annotation using the same coordinates."

While both use normalized 0-1 coordinates, they reference slightly different bounding regions. On a semi-log chart where the y-axis logarithmic scale creates visually varying tick spacing, this difference becomes more pronounced for off-axis positions.

Solutions to achieve proper alignment

Force explicit center anchoring to eliminate the auto-selection variance:

```
javascript
```

```
annotations: [{  
  x: paperX,  
  y: paperY,  
  xref: 'paper',  
  yref: 'paper',  
  xanchor: 'center', // Force center, not auto  
  yanchor: 'middle', // Force middle, not auto  
  textangle: angle,  
  text: label  
}]
```

Apply calculated pixel offsets using `xshift` and `yshift` properties. These apply **after** all positioning and rotation calculations, in screen pixels:

```
javascript
```

```
// Calculate correction based on bounding box expansion
```

```
const W = estimatedTextWidth;
```

```
const H = estimatedTextHeight;
```

```
const radians = angle * Math.PI / 180;
```

```
const expandedW = Math.abs(W * Math.cos(radians)) + Math.abs(H * Math.sin(radians));
```

```
const expandedH = Math.abs(W * Math.sin(radians)) + Math.abs(H * Math.cos(radians));
```

```
// Apply compensating shift
```

```
xshift: (W - expandedW) / 2 * Math.cos(radians),
```

```
yshift: (H - expandedH) / 2 * Math.sin(radians)
```

Use **invisible arrows** to trigger the alternative rotation behavior. With `showarrow: true`, Plotly attaches the anchor point first, then rotates around it—keeping the anchor at your specified position: [GitHub](#) [GitHub](#)

```
javascript
```

```
annotations: [{
```

```
  x: paperX,
```

```
  y: paperY,
```

```
  ax: 0, // Arrow length zero
```

```
  ay: 0,
```

```
  showarrow: true,
```

```
  arrowcolor: 'rgba(0,0,0,0)', // Invisible arrow
```

```
  textangle: angle,
```

```
  text: label
```

```
}]
```

Switch to **domain coordinates** instead of paper coordinates for better alignment between shapes and annotations:

```
javascript
```

```
xref: 'x domain', // Instead of 'paper'
```

```
yref: 'y domain'
```

This is a known, acknowledged Plotly limitation

GitHub Issue #1258 ("annotations aren't plotted properly on log axes") has been **open since December 2016**, tagged P3 backlog and milestone "pre_v2"—indicating it requires breaking changes to fix. The Plotly team has documented that "for log axes, have layout express everything in **data units instead of linearized units**" is planned for V2.0.

Developer Alex Johnson noted in PR #1265: "I don't like it, but I think we need very different behaviors with and without arrows. Without an arrow, when we specify an anchor point as one of the edges or corners... **we do the rotation and THEN set the anchor point based on the new bounding box.**" [GitHub +2](#)

Recommended implementation approach

For semi-log charts with radial annotations, the most reliable solution combines multiple fixes:

1. Set `xanchor: 'center'` and `yanchor: 'middle'` explicitly on all annotations
2. Use the invisible arrow technique (`showarrow: true` with zero-length transparent arrow) to get pre-rotation anchoring
3. If using data coordinates on the log axis, remember to apply `Math.log10()` to annotation positions `Plotly` `Plotly` (shapes don't require this transformation) `github`
4. Consider calculating annotation positions independently from line endpoints, applying empirically-determined corrections for each angle band

The fundamental architectural issue—that shapes and annotations use different coordinate systems and positioning logic—will persist until Plotly V2.0 unifies these behaviors.